Rapport de Projet Traduction des langages

Paul BEDROSSIAN

Arthur BONGIOVANNI

ENSEEIHT

2A - ASR 2024/2025

Sommaire :

Préambule *p2*
partie 1 : pointeurs *p3*
partie 2 : variables globales *p6*
partie 3 : variables statiques locales *p8*
partie 4 : paramètres par défaut *p10*
Conclusion *p12*
Annexes *p13*

Préambule

Ce projet avait pour but de compléter le compilateur de langage Rat vers TAM développé durant les Tp de traduction des langages en ajoutant la possibilité d'utiliser des pointeurs, des variables globale, des variables statiques locales et des paramètres de fonction par défaut.

Ce rapport a pour but d'expliquer les modifications apportées au compilateur pour intégrer les différentes fonctionnalités demandées. Pour chaque partie l'évolution de l'AST et tous les choix d'implémentation seront explicités et justifiés. Enfin, la conclusion servira à discuter des éventuelles autres façons d'implémenter ces fonctionnalités et des améliorations possibles.

Les jugements de typages seront fournis en annexe.

Partie 1 : Les Pointeurs

Pour cette partie, les modifications apportées sont celles vues durant le TD Pointeurs.
Pour prendre en compte les pointeurs, le type affectable est ajouté aux différents Asts. Le type affectable est une abstraction utilisée dans l'Ast pour représenter une entité mémoire qui peut être soit une variable de type t , soit un pointeur vers une telle variable. Cela permet de manipuler uniformément des variables simples et des pointeurs dans le compilateur.

Passe de gestion d'identifiant:

Analyse d'un affectable via la fonction : `analyse_tds_affectable`. Dans cette méthode on analyse l'affectable passé en paramètre selon deux cas de figure :
- l'affectable correspond à un identifiant et est donc un objet que l'on peut analyser comme une variable ou une constante, si l'identifiant n'est pas trouvé par une recherche global, on lèvera une exception "IdentifiantNonDeclare", si le type de l'affectable ne correspond pas, on lèvera une exception "MauvaiseUtilisationIdentifiant")
- l'affectable est un déréférencement et on analyse récursivement l'affectable déréférencé, on remonte ainsi jusqu'à obtenir l'identifiant de l'affectable.

Cette méthode est appelé lorsqu'une expression est de type AstSyntax.Affectable (vérification en lecture) ou qu'une instruction est de type AstSyntax.Affectation (vérification en écriture)
ainsi on s'assure que les pointeurs sont bien identifiés dans le code rat source. Dans le cas d'une expression de type AstSyntax.Addr, soit une adresse, on verifie que l'identifiant est déclaré par une recherche globale puis on vérifie que l'adresse est celle d'une variable.

Passe de typage:

L'analyse se fait via la fonction `analyse_type_affectable`. Comme dans la passe précédente, on regarde selon deux cas :
- dans le cas d'un identifiant, on récupère directement le type de la variable ou un entier dans le cas d'une constante.
- dans le cas d'un déréférencement, on analyse récursivement l'affectable déréférencé et si c'est bien un pointeur, on renvoie le type "Deref of (le type obtenue par l'analyse récursive)" et

l'identifiant.

Comme précédemment, on appelle cette fonction quand on rencontre une instruction de type AstTds.Affectation ou lorsqu'on rencontre une expression de type AstTds.Affectable.

On prend aussi en compte la possibilité d'initialiser un pointeur vers une variable de type t avec l'instruction (New (t)). Cette fonctionnalité est prise en compte avec le type AstTds.New qui sera donc de type pointer of t. De même un pointeur non défini (comme par exemple null) sera de type AstTds.Null.

Pour finir, une expression de type "AstTds.Addr" renverra le type "pointer of t" si elle correspond à une variable de type t, si elle ne correspond pas à une variable, une erreur interne est levée (failwith "Erreur Interne").

Pour s'assurer de la compatibilité des types avec des pointeurs, la fonction "`est_compatible`" vérifie qu'un pointeur peut être assigné à un autre pointeur si leurs types sont compatibles, ou si l'un des deux est un 'pointeur indéfini'.

Passe de placement mémoire:

Dans la passe de placement mémoire, les pointeurs sont traités comme des entiers car ils stockent une adresse, ce qui occupe une seule case mémoire. Aucune modification n'est nécessaire, car la taille et le placement des pointeurs suivent les mêmes règles que pour les types primitifs.

Passe de génération de code:

Tout d'abord, dans le cas d'une expression de type AstType.Addr, on va charger l'adresse d'une variable, donc effectuer un LoadA avec l'offset et le registre correspondant. Ces 2 dernières valeurs sont présentes dans l'infoAst de la variable.

Ensuite, lorsque l'on a une expression de type AstType.Affectable ou une instruction de type Asttype.Affectation, on appelle la fonction "`analyse_mem_affectable`" qui dans le cas d'un identifiant, le traite comme une variable "classique" peut importe son type (si c'est un pointeur, le paramètre de la fonction nécessaire à la récupération de la taille de la variable est le seul changement) et dans le cas d'un déréférencement, l'instruction TAM générer sera le code

nécessaire pour charger la valeur de la variable déréférencé suivi de l'instruction Loadi n ou Storei n (n étant la taille de la valeur à charger) en fonction de si on souhaite un accès en lecture ou en écriture.

Partie 2 : Les Variables Globales

Nous avons décidé pour l'implémentation des variables globales ques celles-ci devait être déclaré au tout début du programme, faisant que la structure d'un programme Rat devient la suivante : (Définition de variable global) list * (Définition de fonction) list * Programme principal. Au niveau de la TDS, cette modification entraîne un potentiel masquage des variables globales par d'autres variables qui seraient définis dans une TDS fille car on recherche localement dans la plupart des cas. Nous avons fait ce choix malgré tout car c'est généralement une mauvaise pratique que de définir des variables avec le même nom qu'une variable globale.
Pour la gestion des variables globales, on ajoute aussi le type Static pour les variables aux différents AST.

Passe de gestion d'identifiant:

Les variables globales sont analysées et ajoutées à la TDS principale avec la fonction "analyse_tds_vars" qui commence par vérifier si la variable n'est pas déjà déclarée (on lèvera une exception "DoubleDeclaration" dans le cas contraire) avant de l'ajouter à la TDS principal et de renvoyer un objet de type AstTds.Static pour caractériser le statut de variable globale.

Passe de typage:

On vérifie la compatibilité entre les types déclarés des variables globales et leurs expressions. Ensuite, on modifie le type de l'identifiant dans l'astTDS et on renvoie un objet de type AstType.Static toujours pour caractériser ce statut globale

Passe de placement mémoire:

On utilise la fonction **"**`analyse_mem_var`**"**pour placer les variables globales dans la pile des variables globales et retourner un objet de type AstPlacement.Static ainsi que la taille de la variable globale. Cette fonction est appelée par "`analyse_mem_vars`" pour générer une liste des variables placés ainsi que la somme de leurs tailles faisant office d'offset général pour la suite de l'analyse du programme.

Passe de génération de code:

Pour chaque variable globale, on va, via la fonction “`var_to_tam`” générer le code pour l’initialiser et la “store” dans le registre “SB” à la suite les unes des autres. Ces lignes de codes seront les premières du code TAM généré par le compilateur.

Partie 3 : Les Variables Statiques Locales

Les variables statiques locales seront analysées comme des variables globales et ont donc le même risque de masquage mais seulement dans les Tds issues de la Tds principale de la fonction. Ce choix à été fait malgré ce défaut car comme pour les variables globales, il n'est pas bonne pratique que d'appeler une variable dans une fonction comme une variable statique locale de cette dernière. On n'autorise pas de déclaration de variable static locales dans le main d'un programme rat car en général, la nécessité d'une variable statique dans le corps du programme principal met en évidence un problème de conception. Les variables statiques locales sont dans la même tds que les paramètres de la fonction pour des soucis d'accès et de visibilité. Elles peuvent masquer des variables globales. étant présente dans une tds fille de la tds principale.
Cette similitude avec les variables globales nous permet une implémentation plus simple et la présence de ces variables dans la même tds que les paramètres de leurs fonction respectives nous permet de limiter leur visibilité et ainsi garantir leurs comportement de variables statiques locales.

Passe de gestion d'identifiant:

Le traitement des variables statiques locales est en soit assez proche de celui des variables globales, ainsi une fois que celles-ci sont détectées dans l'AstSyntax avec le type AstSyntax.StaticLocal, elles sont reconstruites sous formes d'AstSyntax.Static. Cette reconstruction se fait dans la fonction "`split_statics_instr`" (appalé par "`analyse_tds_fonction`") et permet de récupérer deux listes, une pour les variables statiques locales et une autre pour les instructions de la fonction. Les éléments de cette liste de variables statiques sera ensuite analysé comme des variables globales par "`analyse_tds_vars`" mais depuis la tds principale de la fonction. Une fois ce traitement terminé, on renvoie un objet AstTds.Function et la liste des variables statiques locales associées. On répète cette opération sur toutes les fonctions du programme et on obtient une liste de liste de variable statique local reconstruite en variable globale. Ensuite, avant de passer à la passe suivante, on aplati puis

concatène cette liste avec la liste des variables globales, obtenant ainsi une unique liste contenant toutes les variables statiques (globale et locale) du programme. Lorsque qu'on rencontre des instructions de type AstSyntax.StaticLocal, ces dernières étant forcément dans des fonctions (on lève l'exception "`DefStaticMain`" dans le cas d'une déclaration dans le programme principale), on les supprime de la tds de la fonction car elle seront gérées dans les autres passes comme des variables globales.

Passe de typage:

Comme expliqué précédemment, les variables statiques locales étant traitées comme des variables globales, elles sont traitées à la suite de celles-ci de la même manière.

Passe de placement mémoire:

De même que pour la passe précédente, les variables statiques sont analysées comme des variables globales. Il est à noter que l'ajout de variable statique élargit l'offset.

Passe de génération de code:

Toujours comme précédemment, elles ont le même traitement que les variables globales. En conséquence, les variables statique locale seront défini dans le code TAM à la suite des variables globales et sont techniquement considéré comme tel, cependant, leurs utilisations et leurs accès sont contrôlés pour induire un comportement reflétant celui attendu.

Partie 4 : Les Paramètres par défaut

Afin de pouvoir considérer des paramètres par défaut de fonctions, un champ représentant le nombre minimum de paramètres nécessaire à l'appel de la fonction à été ajouté.

Passe de gestion d'identifiant:

La gestion des identifiants des paramètres se fait dans la fonction "`analyse_tds_arg`". Tout d'abord, une fois que l'on s'est assuré de ne pas avoir un paramètre déclarer deux fois dans la signature de la fonction, on l'ajoute dans la tds des paramètres de la fonction. Ensuite, on regarde pour chaque paramètre si une expression lui est rattachée, si il n'y en a pas pour le paramètre, celui ci est "obligatoire" sinon, l'expression analysée constitue la valeur par défaut de ce paramètre. Enfin, on obtient à la suite de ce traitement, une liste de paramètres pour la fonction avec pour chacun, son type, ses informations et son expression s' il en possède). Cette liste est contenu dans un objet de type AstTds.Function obtenu à la suite de l'appel de `analyse_tds_fonction`.

Passe de typage:

Concernant l'analyse des fonctions, on vérifie avec les fonctions "`split3`" (pour séparer la liste des expressions par défaut) et "`analyse_default`" que les paramètres par défaut sont bien les derniers dans la signature et nous renvoie le nombre de paramètres par défaut. On en déduit ainsi le nombre minimum de paramètres à fournir pour l'appel d'une fonction. Ainsi, dans la tds on aura le type de retour de la fonction, la liste des paramètres et le nombre minimum de paramètres.

Concernant maintenant les appels de fonction, on commence par récupérer le type des paramètres fournis. Ensuite on récupère les n premiers paramètre de la fonction (n étant le nombre de paramètre de l'appel). On vérifie ensuite deux choses : en premier que le type des paramètres fournis est compatible avec celui des n premier argument de la fonction. En second, que le nombre de paramètres fourni est compris entre le nombre minimum de paramètres de la fonction et le nombre d'argument total de la fonction. Si tout est correct, on renvoie un

AstType.AppelFonction.

Passe de placement mémoire:

Aucun traitement n’a lieu dans cette passe concernant les paramètres par défaut.

Passe de génération de code:

Lors d’un appel de fonction, on commence par récupérer la liste complète des paramètres de la fonction. À partir du nombre d'arguments fournis, on calcule la taille mémoire des paramètres manquants avec lequel on va générer un Mvoid pour chacun d’entre eux. C'est-à-dire que l’on donne pour chacun des paramètres manquant un “void indéfini” qui sera remplacé dans le début de la fonction par l’expression du paramètre par défaut correspondant. On ajoute aussi, à la suite des Mvoid, un paramètre indiquant le nombre d’argument fourni lors de l’appel qui permettra de savoir combien de paramètres par défaut il faudra initialiser dans le code de la fonction.

Concernant la génération du code de la fonction, on génère un code TAM qui parcourt la liste des paramètres en utilisant un compteur représentant le nombre d'arguments effectivement fournis. Ce compteur se décrémente à chaque paramètre traité. Si ce compteur atteint 0 avant la fin de la liste des paramètres, cela indique la présence de 'Mvoids', qui seront initialisés à l'aide de leurs expressions par défaut

Ce mécanisme garantit une gestion cohérente des paramètres par défaut tout en assurant une séparation claire entre les arguments fournis et ceux automatiquement initialisés.

Conclusion

L'intégration des nouvelles fonctionnalités dans le compilateur a permis d'enrichir les capacités du langage Rat, en y introduisant les variables globales, les variables statiques locales, les paramètres par défaut et leur gestion au sein des appels de fonctions. Chaque étape a nécessité des ajustements significatifs, tant au niveau de l'AST que dans les différentes passes (gestion des identifiants, typage, placement mémoire, et génération de code), afin de garantir la cohérence et l'efficacité de l'ensemble.

Le choix de traiter les variables globales et les variables statiques locales de manière similaire, tout en respectant leurs spécificités, s'est révélé pertinent pour simplifier l'implémentation et minimiser les risques d'erreurs. De même, l'ajout des paramètres par défaut a nécessité des ajustements précis dans la gestion des appels de fonctions, afin de garantir une compatibilité avec les paramètres obligatoires et les valeurs par défaut, tout en maintenant la clarté du code généré.

Cependant, certaines limitations subsistent. Par exemple, les paramètres par défaut sont initialisés de manière séquentielle, ce qui pourrait être optimisé. De plus, les variables globales et statiques, bien que fonctionnelles, pourraient bénéficier d'un traitement séparé pour améliorer la lisibilité du compilateur et faciliter son extensibilité.

En perspective, plusieurs améliorations pourraient être envisagées. L'optimisation de la génération de code pour réduire la taille et le temps d'exécution des programmes compilés constitue une piste prometteuse. Il serait également intéressant d'explorer des approches plus flexibles pour la gestion des paramètres par défaut, ou encore de réfléchir à des techniques permettant de détecter et prévenir plus efficacement les conflits de nommage entre les différentes portées des variables.

En somme, ce projet a permis d'acquérir une compréhension approfondie du fonctionnement d'un compilateur, tout en mettant en pratique des concepts avancés de compilation. Les choix d'implémentation effectués, bien que perfectibles, ont abouti à un résultat fonctionnel et aligné avec les objectifs pédagogiques de ce travail.

Annexes : Jugement de typages

Déréférencement d'un pointeur

$$\frac{\sigma \vdash a : \text{pointeur}(\tau)}{\sigma \vdash *a : \tau}$$

Référencement d'une variable

$$\frac{\sigma \vdash id : \tau}{\sigma \vdash \&id : \text{pointeur}(\tau)}$$

Déclaration d'un pointeur

$$\frac{\sigma \vdash \text{Type} : \tau}{\sigma \vdash \text{new Type} : \text{pointeur}(\tau)}$$

Déclaration d'un pointeur non défini

$$\frac{}{\sigma \vdash \text{null} : \text{pointeur}(\text{Undefined})}$$

Déclaration de variable globale ou de variable statique locale

$$\frac{\sigma \vdash \text{Type} : \tau \quad \sigma \vdash E : \tau}{\sigma,\tau \vdash \text{static Type id} = E : \text{void } [id, \tau]}$$

Déclaration de fonction avec des paramètre par défaut

pas de différence avec le jugement de typage sans paramètre par défaut.

Appel de fonction avec des paramètre par défaut

$$\frac{\sigma \vdash id : \tau1 \to \tau2, \quad \sigma \vdash E1 : \tau1', \ldots, \sigma \vdash Em : \tau m', \quad n-k \leq m \leq n, \quad \forall j \in \{m+1,\ldots,n\}, \sigma \vdash dj : \tau j}{\sigma \vdash id(E1,\ldots,Em) : \tau2}$$

(nb : la fonction prend n paramètres dont k paramètres avec des valeurs par défaut)